University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Lab Report: 05
Experiment tittle: : Implementation of Fork(), Wait(), and Exec() System Calls in Linux using C Programming

Submitted by:

Name : H. M. Tahsin Sheikh

Student ID : 22201243

Section : E1

Submitted to:

Bidita Sarkar Diba

Lecturer,
Department of Computer Science and Engineering

Date of Submission: 13/05/2026

1. Title of the Experiment

Implementation of Fork(), Wait(), and Exec() System Calls in Linux using C Programming

2. Objective

- To learn the use of Linux system calls fork(), wait(), and exec().
- To understand process creation and process management in Linux.
- To execute parent and child processes using C language.
- To observe process synchronization using wait().

3. Introduction

In Linux operating systems, process management is one of the most important concepts. A process is a program in execution. Linux provides several system calls to create and manage processes.

fork() -

The fork() system call is used to create a new process called a child process. After calling fork(), both parent and child processes execute independently.

wait() -

The wait() system call is used by the parent process to wait until the child process finishes execution. It helps in process synchronization.

exec() -

The exec() family of functions is used to replace the current process image with another program. It executes a new command or program inside the current process.

Linux Commands Used:

sudo apt update
sudo apt install build-essential

Create a new C file:

touch new.c

Edit file:

vi new.c

Compile program:

gcc new.c -o x.out

Run program:

./x.out

4. Algorithm

Algorithm for fork() and wait()

1. Start the program.
2. Call fork() to create a child process.
3. If process ID is 0, execute child process statements.
4. Otherwise, execute parent process statements.
5. Parent process calls wait() to wait for child completion.
6. Display process IDs.
7. End the program.

Algorithm for exec()

1. Start the program.
2. Call fork() to create a child process.
3. In child process, call execlp() to execute another Linux command.
4. Parent process waits for child completion using wait().
5. End the program.

5. Source Code

Program 1: Using fork() and wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
   pid_t pid;
   pid = fork();
   if(pid < 0)
   {
      printf("Fork Failed\n");
   }
   else if(pid == 0)
```

```
    {
        printf("Child Process Running\n");
        printf("Child PID: %d\n", getpid());
    }
    else
    {
        wait(NULL);
        printf("Parent Process Running\n");
        printf("Parent PID: %d\n", getpid());
    }
    return 0;
}
```

Program 2: Using exec()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int)getpid());
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        printf("hello, I am child (pid:%d)\n", (int)getpid());
        char *myargs[] = {"wc", "exec.c", NULL};
        execvp(myargs[0], myargs);
        // Only runs if exec fails
        perror("execvp failed");
        exit(1);
    } else {
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
               rc, wc, (int)getpid());
    }
    return 0;
}
```

6. Input/Output

Compilation -
gcc new.c -o x.out
Execution -
./x.out
Sample Output for Program 1 -
Child Process Running
Child PID: 2456
Parent Process Running
Parent PID: 2455
Sample Output for Program 2 -
hello world (pid:2105)

hello, I am child (pid:2106)

32 95 850 exec.c

hello, I am parent of 2106 (wc:2106) (pid:2105)

7. Conclusion

From this experiment, we successfully learned how Linux process management works using fork(), wait(), and exec() system calls. The fork() system call creates a child process, wait() synchronizes the parent and child processes, and exec() replaces the current process with another program. This experiment helped us understand multitasking and process execution in the Linux operating system.